Vietnamese spelling error detection and correction using BERT and N-gram language model

Dong Nguyen Tien[1], Tuoi Tran Thi Minh[1], Loi Le Vu[1,3] and Tuan Dang Minh[1,2]

[1] CMC Institute of Science Technology, No.11 Duy Tan Street, Hanoi, Vietnam,
[2] Posts and Telecommunications Institute of Technology, Km10, Nguyen Trai Street, Hanoi, Vietnam
[3] Hanoi University of Science and Technology, No1. Dai Co Viet st., Hanoi, Vietnam
Corresponding author: tuandm@ptit.edu.vn, dmtuan1@cmc.com.vn
dongnguyentien1996@gmail.com
tuoittm@gmail.com
levuloihust@gmail.com

Abstract. Vietnamese spelling error detection and correction is an important problem in natural language processing and used in many different real-world applications. However, although it has been studied for a long time, there are still many challenges in dealing with different types of errors in Vietnamese. In this paper, we propose a model to help detect and correct some specific Vietnamese spelling errors by combining a pretrained language model for Vietnamese using subword and an n-gram language model. We provide a clear definition of handleable error types, error generation rules in the training set, and evaluate our proposed model on a Vietnamese benchmark dataset at the word level. The experimental results show that our model achieves higher than from 1% to 14% f1 score than others pretrained language models in phrase detection and make comparisons with bi, tri and 4-grams language models to choose the best model for correction.

Keywords: vietnamese spell correction, error detection, deep learning, language model

1 Introduction

Spelling error detection and correction are an important issue and are applied in many systems such as word segmentation [1], named-entity recognition [2], OCR [3] .This is also a problem with great challenges because of the variety of spelling errors and the complexity of the language. To solve the problem, spelling correction systems require a high level of language understanding…

Spell detection and correction aims to detect and convert misspelled words into the correct form in a text of a given language. It has been used in a wide spectrum of natural language applications. For example, spell correction systems can give correction suggestions when there are typos in queries and help information retrieval in search

engines. It can also be incorporated into the existing Automatic Speech Recognition (ASR) or Optical Character Recognition (OCR) to increase the recognition quality.

In this paper, we consider Vietnamese spelling correction in general. Spelling errors are divided into two main categories: non-word errors and real-word errors. Non-word error is an error that doesn't exist in the dictionary, for example, and real-word error is an error that exists in the dictionary, for example canopy. Therefore, a non-word error can be easily identified by using a dictionary to check if a word is in the dictionary, while a real-word error needs to rely on context to determine. Vietnamese spelling error detection and correction is a challenging task for many reasons, such as no explicit word boundaries and polysemous, and relies heavily on contextual semantic information

There have been many studies on spelling correction for other languages such as English, Chinese ... However, to the best of our knowledge, there have not been many studies on spelling correction in Vietnamese. Along with the recent development of language representation models such as BERT to help systems understand languages at a better level, Vietnamese spelling correction is still a problem with great potential in research and development. Therefore, in this work, we introduce a Vietnamese spelling error system consisting of two steps, error detection and error correction, with the main model being using pre-trained language models based on BERT and N- gram language models.

Finally, our contributions are:

- A 2 stage Vietnamese spelling error detection and correction method using both Vietnamese pretrained language model with subword and n-gram language models
- Clarify specific types of errors that can occur in practice and define them specifically. Including those kinds of errors that other method couldn't handle yet
- A public dataset of human-made spelling errors, this dataset is evaluated the proposed method on the various approach

2 Related Word

Several approaches have been proposed for the problem of Vietnamese spelling. Early works followed the pipeline error detection, candidate generation and final candidate selection.

In 2008, PH. Nguyen et al [4] proposed a two-step spelling correction method: error identification and correction. In the detection phase, the author uses dictionaries such as syllable dictionary, proper name dictionary, acronym dictionary to identify non-word errors. For real-word errors, the author uses a syllable bi-gram combining with a POS Bi-gram and some heuristics such as co-occurrence probabilistic, the stopwords list. In the correcting phase, the authors generated candidates by employing heuristics of typical spelling errors and confusion sets based on Hoang Phe dictionary then ranking

candidates using Minimum Edit Distance, SoundEx algorithms and some heuristics such as word lengths, word occurrence frequencies.

Nguyen Thi Xuan Huong et al [5] developed an N-gram language mode for Vietnamese spell correction. A large unlabeled dataset is used to learn the context of syllables. Specifically, the N-gram score for each syllable in the candidate set is calculated based on the frequency of occurrence. The model builds the confusion set for each syllable based on edit distance and chosen confusion set. The current set is considered an error if a syllable in the candidate set has a higher N-gram score than the current one. Another N-gram approach, for Vietnamese OCR post processing, Quoc-Dung Nguyen et al [3] developed an approach for generating and scoring correction candidates based on linguistic features. After the detecting phase, a candidate set for each syllable error will be generated by applying insertion, deletion, and substitution operators. Those papers proved that despite being an old approach, N-gram is still a powerful model to solve natural language processing problems with impressive performance.

H. Nguyen, 2015 [6] built a tool to normalize Vietnamese tweets on Twitter, in which the author proposes a spelling correction model consisting of 2 steps: error identification and correction. The author builds a dictionary of Vietnamese words and defines a word as an error if the word does not exist in the dictionary. For each error, the author used the set of syllable rules to normalize, then the result will be input to the next phase to measure the similarity with the word in the dictionary using dice coefficient to find words with the highest similarity degree. Moreover, the above 2-step model shows practicality, when many systems want to maximize error detection and suggest possible cases without auto-correction. Therefore, with the recent development of deep learning, pre-trained neural language models can apply to spelling error detection, correction and yield good results due to its better language representation.

Most recently, H Tran et al., 2021 [7] proposed a Hierarchical Transformer model for Vietnamese spelling correction. The model uses two Transformer encoders: Character-level encoder and Word-level encoder to represent words on two levels of words and characters. Then the word representation will go through the detection classifier to determine if a word is misspelled or not and finally a correction classifier to suggest the replacement word for the misspelled word. Each classifier consists of two fully connected layers followed by the softmax activation function on the last layer. However, the seq2seq model has a common disadvantage that the performance cannot be as good as the token classification model. Moreover, errors that are not at the word level such as word stickiness, the model cannot handle it

3 The proposed model

3.1 Problem formulation

Vietnamese spelling error detection and correction (VSEDC) is formulated as follows: Given a sentence of n words $X = (x_1, x_2, \dots, x_n)$, VSEDC aims at transforming X into

a new sentence X' of m words $X' = (x'_1,\ x'_2, \dots, x'_m)$, with $m\ \geq n$. Specifically, each misspelled word in the input sentence X is replaced with one or more correct words in the output sentence X'.

In this paper, we proposed a model to solve 3 following types of error:

- **MISSING_SPACE**: As the name suggests, this error happens when 2 consecutive words are sticked together, example: *chính tả - chínhtả*. Another example for this error is chính tả - chinhstar, which happens very regularly when typing. We consider this error type as non-word error since we observed that there is no word of this error type appearing in Vietnamese vocabulary. A word of **MISSING_SPACE** error will be replaced by at least 2 correct words.

- **EDITS:** This error happens when a unit in a word is altered, insert, delete, replace and permute, where a unit is defined as a vowel (exclude of accents), i.e: *a, e, i, o, u, y or a* consonant (include 2 or 3-letter consonants), i.e: *b, c, ngh, gi,* ... For example, *sửa lỗi - xửa lỗi, chính tả - thính tả*. For this type of error, we only consider real-word error since non-word error can be easily addressed by checking the vocabulary.

- **MISSING_ACCENT:** This error type happens when a vowel misses its accents, e.g: *sửa lỗi - sua lỗi, chính tả - chinh ta*. MISSING_ACCENT is separated from EDITS in error correction phase since it includes cases in which there are more than 1 word being misspelled. For this type of error, we consider 50% as real-word error and 50% as non-word error.

3.2 Data Prepocessing

A characteristic of Vietnamese language is the appearance of accents, e.g*: á, à, ả, ã, ạ, â, ă,* ... These letters are not available on the keyboard. Instead, they are combinations of several other letters as defined by Telex input method, e.g., *as - á, af - à, ar -* ả. We call such combinations *(as, af, ar)* telex form and the results displayed by these combinations *(á, à, ả)* accent form. Because of using the Telex input method, there likely exist letters in telex form when people make mistakes in typing. Hence, before detecting and correcting errors, we replace all occurrences of telex form by their corresponding accent form.
However, in the error correction phase, before generating candidates for **EDITS** error, we convert words in accent form to telex form. This is due to 3 reasons: (1) misspelled words of non-word error usually appear in telex form instead of accent form, for example: *suaawr lỗi, chính tarr* instead of *sửaa lỗi chính tảr*; (2) The number of characters to be considered is much smaller in telex form than in accent form. Specifically, in accent form, there are 67 letters with diacritics and tones, including 7 diacritical letters *ă, â, ê, ô, ơ, ư, đ* and 60 letters that are combinations of 12 vowels *a, ă, â, e, ê, i, o, ô, ơ, u, ư, y* and 5 tones (*á, à, ã, ả, ạ*), not appearing in telex form. Whereas there are only 3 letters f, j, w in telex form that do not appear in accent form; (3) In case there are 2 vowels in a word, mistyping a single button on keyboard can lead to misspelling of 2

letters in accent form, *trường học vs truòng học*. In this example, there is a missing w for the word truòng to be corrected, i.e: *truongfw* - *trường*, *truongf* - *truòng*. In this case, converting accent form to telex form helps reduce 2 misspelled letters to only 1 misspelled one. We also define several rules when converting accent form to telex form, since there might be more than one telex form for an accent form, e.g: chính tả can be typed as *chinhs tar* or *chisnh tar*.

3.3 Model

We proposed a spell correction model including 2 phases: error detection and error correction. Figure 1 is the illustration of our proposed model. As shown in **Fig. 1.** Overview of Vietnamese spelling error detection and correction (VSEDC), **MISSING_SPACE** errors are detected and corrected first and after that are **EDITS** and **MISSING_ACCENT** errors. Error detection can be viewed as a sequential labeling problem, takes as input a sequence of n words *(*)* $X = (x_1, x_2, \dots, x_n)$ and assigns each word a label **0** for the correct word and **1** for misspelled word. Error correction takes the sequence X and labels of words in X' produced by error detection phase, then generates candidates for misspelled words and ranks those candidates using n-gram language model. *(*)* we only use single syllable words because sentences with misspelled words might be incorrectly segmented.

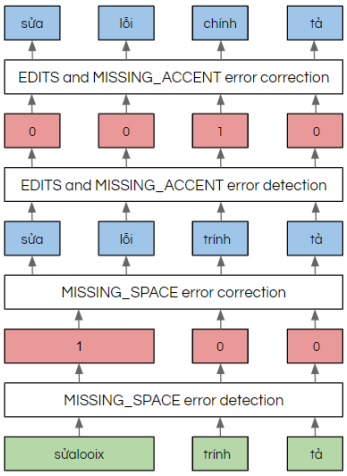

Fig. 1. Overview of Vietnamese spelling error detection and correction (VSEDC)

3.4 Error Detection

Input of error detection is a sequence of n words $X = (x_1, x_2, \dots, x_n)$, and output is a sequence of n labels $Y = (y_1, y_2, \dots, y_i, \dots, y_n), y_i \in \{0, 1\}$, where y_i is the label of word x_i. For each type of error, we use a different method for detection. To be specific: For **MISSING_SPACE**, we use regular expression. We defined a regex pattern for detecting **MISSING_SPACE** errors based on the observation that there are no words

in Vietnamese in the format vowel - consonant - vowel. A word is labeled as **1** if it matches the pattern and **0** otherwise. If there exists any word in telex form, we convert them to accent form before error detection.
We built a Vietnamese pre-trained language model based on BERT called CMCBert using a large unlabeled Vietnamese corpus. This corpus contains 25 GB articles in the following disciplines: health, education, information technology, … 1 GB Vietnamese Wikipedia, and 1 GB Facebook status. CMCBert constructs word representations at the subword level using sentencepiece tokenizer. In this work, we fine-tuned CMCBert for binary token classification task on a small amount of data which is annotated for the purpose of detecting EDITS and **MISSING_ACCENT** errors. Token classifier classifies a subword into one of the two classes: 1 for **EDITS** and **MISSING_ACCENT** error and 0 otherwise. A word is defined as being misspelled if at least one of its subwords has label 1. Details of our model is shown in **Fig. 2.** Error detection model for **EDITS** and **MISSING_AGENT** errors.

BERT [8] is a bi-directional pretrained language model that bases on Transformer architecture. Transformer includes two main components, which are encoder and decoder, where each component is a stack of several identical layers. Each layer in encoder contains 2 sublayers, namely multi-head self-attention and position-wise fully connected feed-forward. Each layer in decoder also contains these two sublayers, but has an additional masked multi-head self-attention sublayer to guarantee that predictions for a position *i* can depend only on the known outputs at positions less than *i*. This is done by removing positions greater than *i* from attention mechanism. Multi-head self-attention are defined as follows:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^0 \quad (1)$$

$$where\ head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (2)$$

$$FFN(X) = \max(0, XW_1 + b_1)W_2 + b_2 \quad (3)$$

We denote the sequence of hidden states at the final layer of BERT as $H = (h_1, h_2, \dots h_n)$. Then, the probability of a word x_i belonging to class $i, j \in \{0, 1\}$ is defined as:

$$p_{ij} = P(y_i = j|X) = softmax(Wh_i + b) \quad (4)$$

Q, *K*, *V* are the same matrices representing the input sequence or the output of the previous layer $W^o, W_i^Q, W_i^K, W_i^V, W_1, b_1, W_2, b_2, W, b$ are parameters and objective function is cross-entropy loss:

$$L = \frac{-1}{N}\sum_{i=0}^{N}\sum_{j\in\{0,1\}} y_{ij} \log p_{ij} \quad (5)$$

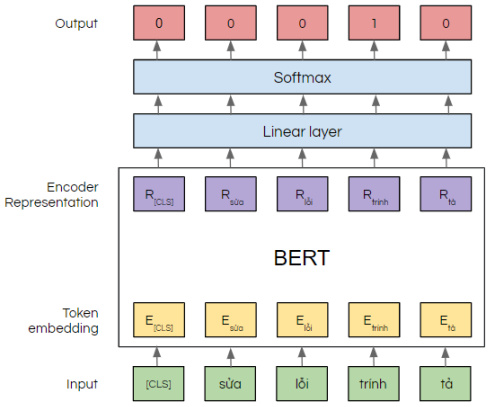

Fig. 2. Error detection model for **EDITS** and **MISSING_AGENT** errors

3.5 Error Correction

Input of error correction is a sequence of n words $X = (x_1, x_2, \dots, x_n)$ and n corresponding labels $Y = (y_1, y_2, \dots, y_i, \dots, y_n), y_i \in \{0, 1\}$. The output is a sequence of m words that are already corrected $X = (x'_1, x'_2, \dots, x'_m), m \geq n$. As aforementioned, **MISSING_SPACE** errors are corrected before detecting and correcting the other two error types. We build a vocabulary V including telex form of all single-syllable words in Vietnamese. This vocabulary is used to check if a word exists, where the existence of a word is defined as the appearance of the that word in the vocabulary.

Error correction includes the following steps. For each misspelled word, a set of candidates is generated and then scored using n-gram language model. Score of a candidate is defined as the score of the sentence containing that candidate. After that, candidates are ranked by scores and top k candidates after ranking are selected. We set $k = 1$ for **MISSING_SPACE** error type and $k = 10$ for **EDITS** and **MISSING_ACCENT**. After completing error correction for **EDITS** and **MISSING_ACCENT**, we evaluate our model using top 1, 2, 3 candidates.

N-gram language model [9] is a probabilistic language model that assigns probability for a sentence based on Markov assumption, meaning that probability of a word depends only on N - 1 previous words. Given a sequence $P = (w_1, w_2, \dots, w_n)$, the general equation for N-gram approximation to the conditional probability of the next word in a sequence is defined as:

$$P(w_n | w_{1:n-1}) \sim P(w_n | w_{n-N+1:n-1}) \tag{6}$$

Score of the sequence P is defined as:

$$P(w_{1:n}) = \prod P(w_k | w_{k-N+1:k-1}) \tag{7}$$

We built a trigram language model ($N = 3$) using KenLM [10] language model library. For each error type, we use different candidate generation methods.

For **MISSING_SPACE** error type, candidates are generated by separating the error word into a pair of subwords at positions j, j+1, ..., where j is the position of the first vowel in that word. Pairs which contain at least one subword existing in the vocabulary will be kept. This procedure is repeated until all MISSING_SPACE errors are removed.

For **EDITS** error type, before candidate generation, we convert words into their telex form. Candidates are generated by inserting, deleting a unit, substituting a unit by another one or permuting positions of 2 consecutives units. Then candidates that exist in the vocabulary will be kept. If there are no valid candidates, we keep the original misspelled word.

For **MISSING_ACCENT** error type, we generate candidates by adding diacritics or tones to the vowels of misspelled words. Like **EDITS**, we only keep candidates that exist in the vocabulary and if there are no valid candidates, we keep the original word.

After completing candidate generation, we convert those candidates to accent form as the results.

4 Experiments and results

4.1 Data Preparation

We built a dataset named ARTICLES containing 12,300,000 sentences collected from articles in Health Services, Education, Information Technology … This dataset contains no errors, so we generated errors in the following ways. The MISSING_SPACE error is generated by removing space between random adjacent words and converting one or some of them into telex form. For a random word, the EDITS error is caused by insertions, deletions, or substitutions of a single character or transposition/permutation of two adjacent characters, the MISSING_ACCENT error is obtained by removing all accent marks of vowels. The proportions of non-word error and real-word error of error types are described in **Table 1**.

Table 1. The proportions of non-word error and real-word error of each type of error.

	MISSING_SPACE	**EDITS**	**MISSING_ACCENT**
Non-word error	100%	0%	0%
Real-word error	0%	100%	100%

This dataset was then divided into three datasets ARTICLES-KENLM, ARTICLES-TRAIN and ARTICLES-TEST. The ARTICLES-TRAIN contains 1,600,000 sentences used to build an Error Detector in the Error Detection step. The ARTICLES-KENLM contains 10,000,000 sentences used to build a language model to rank the candidates in the Error Correction step. We performed evaluations on the ARTICLES-TEST containing 700,000 sentences.

The error rate per sentence and the total number of errors in the training and test datasets are shown in **Table 2** and **Table 3** respectively.

Table 2. The average error rate per sentence in the training and test dataset

	ARTICLES-TRAIN	ARTICLES-TEST
MISSING_SPACE	0%	20%
EDITS	20%	20%
MISSING_ACCENT		20%

Table 3. The total number of errors in the training and test dataset

	ARTICLES-TRAIN	ARTICLES-TEST
MISSING_SPACE	0	0
EDITS	2365995	2365995
MISSING_ACCENT	3726560	3726560

4.2 Experiments settings

In fine-tuning of CMCBert for training Error Detector, we kept the default hyper-parameters and only fine-tuned the parameters using Adam optimizer. We use the learning rate scheduler with the warm-up step of **0.1** and the learning rate of **2e-5**. The model is trained for 300,000 steps (3 epochs). We also built a 2, 3, and 4-grams language model using the Kenlm for ranking candidates in the Error Correction step.
We utilized recall, precision, and F1-score metrics to evaluate Error Detector performance as in most classification tasks. For Error Corrector, we used two metrics w2t and t2w are defined as follows:

$$w2t = \frac{c_{w2t}}{c_w} \tag{8}$$

$$t2w = \frac{c_{t2w}}{c_t} \tag{9}$$

c_{w2t} is the number of errors is converted into the correct form, c_{t2w} is the number of non-errors unexpectedly converted into the wrong form, c_w is the total number of errors, and c_t is the total number of non-errors.

4.3 Results

We performed evaluations on 700,000 sentences in the ARTICLES-TEST dataset. First, we individually evaluate the Error Detection step and Error Correction step, which means we use ground truth as the input for the Error Correction. The results of the Error Detection step and the Error Correction step are shown in **Table 4** and

Table 4. Evaluation results of Error Detection. F1-score is reported for our pre-trained language model, XLM-R base and Phobert base

	CMCBert	**Phobert**[11]	**XLM-R**[12]
MISSING_SPACE	0.95		
EDITS	0.95	0.94	0.81
MISSING_ACCENT	0.97	0.97	0.83

As shown in **Table 4.** Evaluation results of Error Detection. F1-score is reported for our pre-trained language model, XLM-R base and Phobert base, our model performs slightly better than Phobert at **MISSING_ACCENT** error. Additionally, CMCBert does not require word segmentation when preprocessing data, thus significantly reducing inference time when applying in practice. CMCBert and Phobert greatly outperform the XLM for both **EDITS** and **MISSING_ACCENT** errors. This suggests that monolingual language model is much better than multilingual one in monolingual Spelling Correction task.

Table 5. Evaluation results of Error Correction, ground truth is used as input of Error Correction step, so we left out the t2w metric

	w2t		
	N=2	N=3	N=4
MISSING_SPACE	0.99	0.99	0.99
EDITS	0.83	0.89	0.86
MISSING_ACCENT	0.78	0.89	0.95

Table 5. Evaluation results of Error Correction, ground truth is used as input of Error Correction step, so we left out the t2w metricshows that 4-grams language model is better than 2 and 3-grams on average. Since 4-grams model can capture longer distance dependency than the other two. N grams language model can perform better than 1-gram but it requires much more data to build and its space complexity grows by an order of magnitude with base N. So, we select 4-grams language model.

Table 6. Evaluation results on combined Error Detection and Error Correction steps, that is, the output of the Error Detection step is the input of the Error Correction step. The pre-trained language model CMCBert and the 4-grams language model are used for this evaluation.

	w2t	t2w
MISSING_SPACE	0.93	0.93
EDITS	0.01	0.01
MISSING_ACCENT	0.83	0.83

We report end-to-end results in **Table 6.** Evaluation results on combined Error Detection and Error Correction steps, that is, the output of the Error Detection step is the input of the Error Correction step. The pre-trained language model CMCBert and the

4-grams language model are used for this evaluation.. The results indicate that our proposal achieves high accuracy in error correction while minimizing the number of words unexpectedly converted to wrong form. The rate of **EDITS** errors corrected is the lowest because of its diversity, i.e, a word can be converted into many forms of **EDITS** errors.
Finally, we demonstrated several examples of misspelled sentences and their correct forms after using our model

Table 7. Examples of misspelled sentences and corresponding correct sentences.

Misspelled sentence	Corrected sentence
Nhiều giải pháp đã áp dụng **thành-coong** trong thực tiễn, có **doanhthu** tương đối lớn, phục vụ **khoongchir** cho doanh nghiệp mà còn cho đời sống xã hội	Nhiều giải pháp đã áp dụng **thành công** trong thực tiễn, có **doanh thu** tương đối lớn, phục vụ **không chỉ** cho doanh nghiệp mà còn cho đời sống xã hội
Nhiều **giãi** pháp đã áp dụng thành công trong thực tiễn, có doanh thu tương **dối** lớn, phục vụ không **chỏ** cho doanh nghiệp mà còn cho đời **sóng** xã hội	Nhiều **giải** pháp đã áp dụng thành công trong thực tiễn, có doanh thu tương **đối** lớn, phục vụ không **chỉ** cho doanh nghiệp mà còn cho đời **sống** xã hội

5 Acknowledgement

We gratefully acknowledge the support from the CMC Institute of Science Technology for funding the research project.

6 Conclusion

The investigation in this paper presented Vietnamese spelling error detection and correction using BERT and N-gram language model. The experiment demonstrated that the proposed method takes advantage of both pretrained and n-gram language model to make predictions 3 type of errors missing space, edits and missing accent. The model trained on 12,300,000 sentences randomly generate form large corpus of high-quality resources. Although working well with our expectations, the model does not generally solve the others spelling problem. For future work, we plan to expand the type of error spelling. We also want to design more robust and powerful architectures that allow us to offer a more satisfying experience to users.

References

1. H. Thixuan Nguyen, T. Dang, C. Anh Le, H. ThiXuan Nguyen, T.-T. Dang, and A.-C. Le, "Adapting Vietnamese Word Segmentation for Microblogs-Style Data," 2014. [Online]. Available: https://www.researchgate.net/publication/283502832.
2. V. H. Nguyen, H. T. Nguyen, and V. Snasel, "Named entity recognition in Vietnamese tweets," *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 9197, no. August, pp. 205–215, 2015, doi: 10.1007/978-3-319-21786-4_18.
3. Q. D. Nguyen, D. A. Le, and I. Zelinka, "OCR error correction for unconstrained Vietnamese handwriten text," *PervasiveHealth Pervasive Comput. Technol. Healthc.*, pp. 132–138, 2019, doi: 10.1145/3368926.3369686.
4. P. H. Nguyen, T. D. Ngo, D. A. Phan, T. P. T. Dinh, and T. Q. Huynh, "Vietnamese spelling detection and correction: Using Bi-gram, minimum edit distance, SoundEx algorithms with some additional heuristics," *RIVF 2008 - 2008 IEEE Int. Conf. Res. Innov. Vis. Futur. Comput. Commun. Technol.*, pp. 96–102, 2008, doi: 10.1109/RIVF.2008.4586339.
5. H. T. Nguyen, T. B. Dang, and L. M. Nguyen, "Deep Learning Approach for Vietnamese Consonant Misspell Correction," *Commun. Comput. Inf. Sci.*, vol. 1215 CCIS, no. November, pp. 497–504, 2020, doi: 10.1007/978-981-15-6168-9_40.
6. V. H. Nguyen, H. T. Nguyen, and V. Snasel, "Normalization of Vietnamese tweets on Twitter," *Adv. Intell. Syst. Comput.*, vol. 370, no. June, pp. 179–189, 2015, doi: 10.1007/978-3-319-21206-7_16.
7. H. Tran, C. V. Dinh, L. Phan, and S. T. Nguyen, "Hierarchical Transformer Encoders for Vietnamese Spelling Correction," May 2021, [Online]. Available: http://arxiv.org/abs/2105.13578.
8. J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *NAACL HLT 2019 - 2019 Conf. North Am. Chapter Assoc. Comput. Linguist. Hum. Lang. Technol. - Proc. Conf.*, vol. 1, no. Mlm, pp. 4171–4186, 2019.
9. J. Lin, "N-Gram Language Models N-Gram Language Models," 2009.
10. Kenneth Heafield, "KenLM: Faster and Smaller Language Model Queries," *Proc. Sixth Work. Stat. Mach. Transl.*, pp. 187--197, 2011.
11. D. Q. Nguyen and A. Tuan Nguyen, "PhoBERT: Pre-trained language models for Vietnamese," pp. 1037–1042, 2020, doi: 10.18653/v1/2020.findings-emnlp.92.
12. S. Ruder, A. Søgaard, and I. Vulic, "Unsupervised cross-lingual representation learning," *ACL 2019 - 57th Annu. Meet. Assoc. Comput. Linguist. Tutor. Abstr.*, pp. 31–38, 2019, doi: 10.18653/v1/p19-4007.